

Fairness among TCP connections

Project in Computer Networks - 236340 - Spring 2025

Team: Alaa Abo Mokh, Adan Wattad, Adan Jbara

Advisors: Eran Tavor

Table Of Contents

Introduction

Background

Congestion Control Protocols

- TCP Fundamentals
- TCP NewReno
- CUBIC
- DCTCP
- BBR

Fairness Metrics

- Jain's Index
- Gini (G) Index
- Max-Min Fairness

Experimental Setup

- Dumbbell Topology and Bottleneck
- Environment Types (Single / Mixed Protocols)
- RTT Conditions (Symmetric / Asymmetric)

Results and Analysis

- Single-Protocol Results (Reno, CUBIC, DCTCP, BBR)
- Mixed-Protocol Fairness (Protocol A vs Protocol B)
- Cross-Comparison and Discussion of Findings

Conclusions and Future Work

Objectives and Project Plan

- Goals & Estimated Timetable

References & Appendix

Introduction

General Description

Modern cloud computing relies heavily on datacenters, which form the backbone for delivering scalable and cost-effective services to millions of users worldwide. Within these datacenters, thousands of servers are interconnected through high-speed networks to support diverse applications, ranging from interactive user services to large-scale data processing. The performance of these networks directly impacts the reliability and quality of the services provided.

One of the main challenges in such environments is managing network congestion. Congestion control protocols are designed to regulate traffic flow and ensure efficient use of resources, especially as multiple connections compete for bandwidth. The choice of protocol can significantly influence throughput, latency, and the overall user experience.

For decades, TCP (Transmission Control Protocol) has been the dominant transport protocol in both the Internet and datacenter networks. TCP provides reliable data delivery and incorporates congestion control mechanisms that adapt to network conditions. Its widespread use and central role in networked systems make it essential to study and evaluate TCP behavior under various network scenarios.

This project focuses on studying TCP connections under different network topologies using the NS-3 network simulator. The work begins with experiments on a simple dumbbell topology, which provides a controlled environment for testing. This project specifically examines fairness as a criterion for comparison between different congestion control protocols within the commonly used dumbbell topology. By simulating TCP traffic under this scenario, the project aims to provide valuable insights into protocol behavior and performance within datacenter infrastructures.

Background

NS-3:

NS-3 is a powerful and widely used discrete-event network simulator designed for research and educational purposes. It provides a comprehensive platform for modeling and analyzing network protocols, topologies, and applications in a controlled and reproducible environment.

For this project, NS-3 was used to simulate different TCP congestion control protocols, allowing a detailed examination of their performance under the dumbbell topology. This setup makes it possible to evaluate how various algorithms behave when multiple connections share the same network resources.

To structure the analysis clearly, we begin by presenting the theoretical background of the congestion control protocols studied in this project. These sections cover their key principles, mechanisms, and algorithmic behavior, establishing the foundation for our simulations and guiding the evaluation of fairness across different TCP variants in NS-3.

Congestion Control Protocols

Transmission Control Protocol (TCP):

Transmission Control Protocol (TCP) is a connection-oriented communication protocol that ensures the reliable exchange of data between devices across a network. It operates at the transport layer (Layer 4) of the OSI model and works together with the Internet Protocol (IP) forming the well-known TCP/IP suite that powers most of the internet.

TCP is designed to provide error-free, ordered, and complete delivery of data. Before data transfer begins, TCP establishes a connection using the three-way handshake (SYN, SYN-ACK, ACK). Similarly, it closes the connection using a four-step process (FIN, ACK, FIN, ACK) to ensure both ends disconnect properly.

How TCP Works?

When an application sends data, TCP breaks it into small units called segments. Each segment is assigned a sequence number to keep track of its position in the stream. On the receiving side, TCP reassembles the segments in order, ensuring no data is lost or duplicated.

If a segment is corrupted or lost, the receiver requests retransmission using acknowledgments (ACKs). TCP also adapts the data transmission rate according to the receiver's buffer size and the current network conditions, which helps avoid congestion.

Key Features

- **Connection-Oriented:** A logical connection is maintained between sender and receiver until communication is complete.
- **Reliable Delivery:** Ensures data is delivered correctly and in the same order it was sent.
- **Full Duplex Communication:** Data can be sent and received simultaneously.
- **Flow Control:** Uses the sliding window technique to control the rate of data transmission and prevent overflow at the receiver.

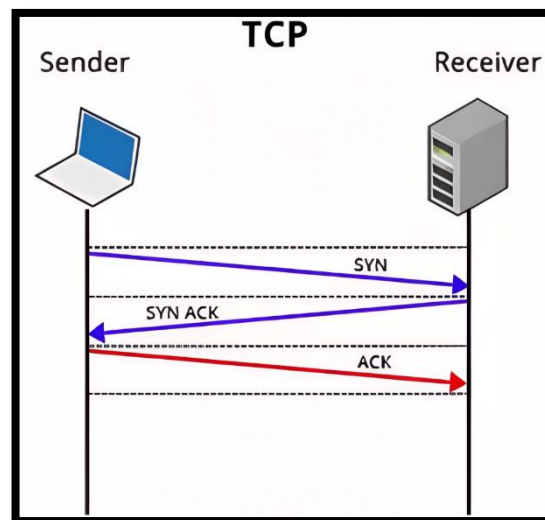
- Error Control: Detects and corrects transmission errors using checksums and retransmissions.
- Congestion Control: Manages network load through algorithms such as Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery.

Advantages

- Provides reliable and ordered delivery of data.
- Supports error detection, correction, and retransmission.
- Adapts automatically to network conditions.
- Standardized and widely supported across all modern networks.

Disadvantages

- Has higher overhead and slower performance compared to simpler protocols like UDP.
- Not ideal for small or low-resource networks.
- Works only within the TCP/IP framework (not suitable for non-IP networks like Bluetooth).



Congestion Control:

Congestion control ensures efficient data flow in a network by preventing overload, minimizing latency, and maximizing throughput. It dynamically adjusts the congestion window (cwnd) or packet transmission rate based on feedback mechanisms such as packet loss, delay, or Explicit Congestion Notification (ECN). By regulating the sender's transmission behavior, congestion control maintains network stability, prevents excessive queuing, and ensures fairness among competing flows.

TCP CUBIC:

CUBIC is a network congestion avoidance algorithm for TCP which can achieve high bandwidth connections over networks more quickly and reliably in the face of high latency than earlier algorithms. It helps optimize long fat networks.

Characteristics:

CUBIC is a less aggressive and more systematic derivative of BIC TCP, in which the window size is a cubic function of time since the last congestion event, with the inflection point set to the window size prior to the event. Because it is a cubic function, there are two components to window growth. The first is a concave portion where the window size quickly ramps up to the size before the last congestion event. Next is the convex growth where CUBIC probes for more bandwidth, slowly at first then very rapidly. CUBIC spends a lot of time at a plateau between the concave and convex growth region which allows the network to stabilize before CUBIC begins looking for more bandwidth.

Another major difference between CUBIC and many earlier TCP algorithms is that it does not rely on the cadence of RTTs to increase the window size. CUBIC's window size is dependent only on the last congestion event. With earlier algorithms like TCP

New Reno, flows with very short round-trip delay times (RTTs) will receive ACKs faster and therefore have their congestion windows grow faster than other flows with longer RTTs. CUBIC allows for more fairness between flows since the window growth is independent of RTT.

Algorithm:

CUBIC increases its window to be real-time dependent, not RTT dependent like BIC. The calculation for cwnd (congestion window) is simpler than BIC, too.

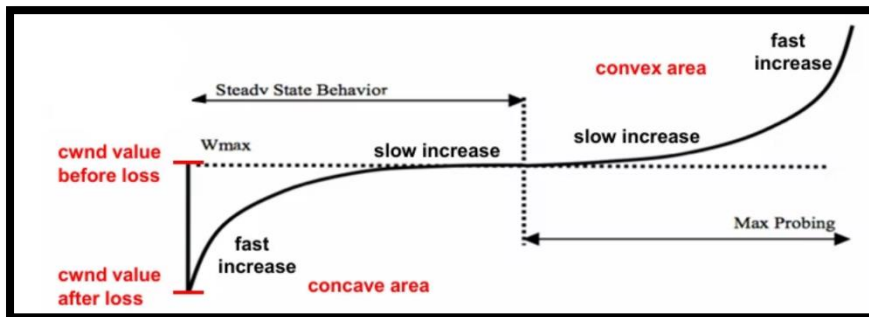
Define the following variables:

- β : Multiplicative decrease factor
- w_{max} : Window size just before the last reduction
- T : Time elapsed since the last window reduction
- C : A scaling constant
- $cwnd$: The congestion window at the current time

RFC 8312 indicates the following:

- The unit of all window sizes in this document is segments of the maximum segment size (MSS), and the unit of all times is seconds. (Section 4)
- β SHOULD be set to 0.7 (Section 4.5)
- C SHOULD be set to 0.4 (Section 5)

Then $cwnd$ can be modeled by:



$$cwnd = C(T - K)^3 + w_{max}$$

$$\text{where } K = \sqrt[3]{\frac{w_{max}(1-\beta)}{C}}$$

TCP BBR (Bottleneck Bandwidth and Round-Trip Propagation Time):

BBR is a modern congestion control algorithm developed by Google. Unlike traditional TCP algorithms (like Reno and NewReno), which rely on packet loss to detect congestion, BBR uses a model-based approach that measures the bottleneck bandwidth (BtlBw) and round-trip propagation time (RTprop) to decide the optimal sending rate.

Motivation:

Traditional loss-based algorithms assume packet loss signals congestion. This works with small router buffers, but fails in today's networks:

- In shallow-buffer networks, packet loss may occur even when there is no real congestion, leading to underutilization.
- In deep-buffer networks, loss-based control fills buffers unnecessarily, causing high latency (bufferbloat).

BBR overcomes these issues by responding to actual delivery performance instead of packet drops.

Core Idea:

BBR builds a dynamic model of the connection using two parameters:

- BtlBw (Bottleneck Bandwidth): Maximum recent delivery rate.
- RTprop (Round-Trip Propagation Time): Minimum recent RTT observed.

It then sets the congestion window according to the Bandwidth-Delay Product (BDP):

$$Cwnd = BtlBw \times Rtp\text{prop}$$

This ensures the sender keeps just enough data “in flight” to fully utilize the link while keeping latency low.

BBR Operation Phases

1. Startup:

BBR quickly probes available capacity, doubling its sending rate each RTT (similar to TCP Slow Start). It ends startup when throughput stops growing significantly (less than $1.25\times$ over three RTTs).

2. Drain:

After queues build up, BBR reduces its pacing rate to drain excess data, lowering inflight data to roughly one BDP.

3. Steady State (Probe Bandwidth):

BBR alternates between increasing and decreasing its sending rate by about 25% for one RTT each, testing whether bandwidth has changed.

- If delivery rate improves $\rightarrow$ adopt higher rate.
- If not $\rightarrow$ reduce rate to maintain stability.

Periodically, BBR also checks if RTT has increased and briefly drains queues to re-estimate RTTprop.

Key Features

- Model-based: Uses bandwidth and delay, not packet loss.
- Adaptive: Continuously updates BtlBw and RTTprop to match network conditions.
- Efficient: Achieves high throughput with minimal queuing delay.
- Fairness: Aims to share bandwidth fairly with both BBR and non-BBR flows.

Conclusion

BBR represents a new generation of congestion control focused on rate-based modeling rather than packet loss. By explicitly estimating the available bandwidth and propagation delay, it achieves higher throughput, lower latency, and better overall network efficiency than traditional TCP algorithms.

TCP NewReno:

TCP NewReno is an improvement over TCP Reno, designed to handle multiple packet losses within a single congestion window more efficiently.

It builds upon TCP Tahoe and Reno, which introduced congestion control mechanisms such as Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery.

In TCP Reno, when packet loss occurs, the sender halves its congestion window (cwnd) and enters fast recovery. However, Reno struggles when multiple packets are lost in one window, reducing cwnd multiple times unnecessarily and taking longer to recover.

Limitations of TCP Reno

- Detects multiple packet losses slowly.
- Reduces cwnd repeatedly for losses in the same window, lowering throughput unnecessarily.

How TCP NewReno Improves Reno

NewReno introduces the concept of partial acknowledgments (partial ACKs) to overcome these issues.

When the sender receives an ACK for a retransmitted packet, it checks whether it acknowledges all data sent before the loss.

- If all data in that window is acknowledged → it's a full ACK, and TCP exits fast recovery.
- If some data is still missing → it's a partial ACK, meaning more losses remain, so the sender stays in fast recovery and retransmits the next missing segment without reducing cwnd again.

This prevents premature exit from fast recovery and allows TCP to recover multiple losses smoothly within one congestion window.

NewReno's Fast Retransmit & Recovery

Key mechanisms:

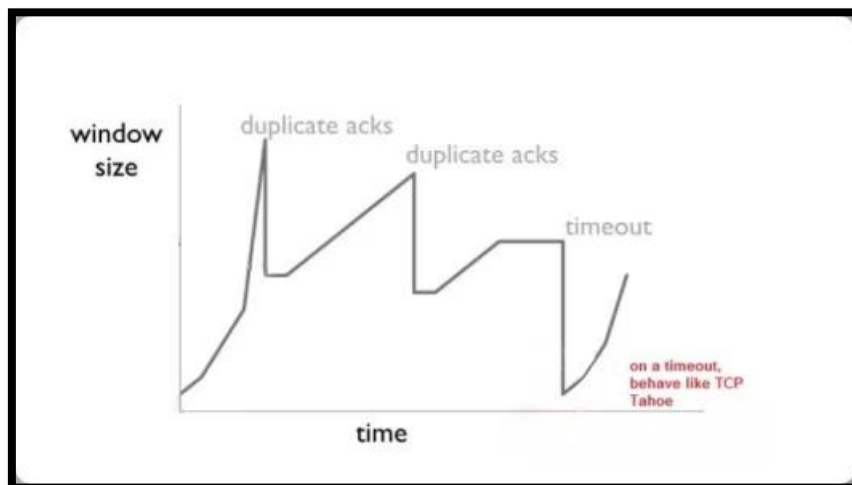
- Duplicate ACKs: After 3 duplicates, loss is inferred, triggering fast retransmit.
- Recover Variable: Tracks the highest sequence number sent when recovery begins.
- Partial ACKs: Retransmit the next missing segment and stay in recovery.
- Full ACK: All data recovered — exit fast recovery and reset cwnd.
- Timeouts: Exit recovery and reset variables if retransmit timeout occurs.

Advantages of NewReno

- Handles multiple packet losses without repeated cwnd reductions.
- Improves throughput and stability compared to Reno.
- Works well even without SACK (Selective Acknowledgment) support.
- Smooths congestion recovery and reduces unnecessary timeouts.

Conclusion

TCP NewReno enhances TCP's reliability by improving how losses are detected and recovered within a single window. It remains one of the most practical and widely used TCP variants due to its simplicity, efficiency, and compatibility with both SACK and non-SACK systems.



DCTCP (Data Center TCP):

DCTCP (Data Center TCP) is a variant of TCP optimized for data center environments, where links have high bandwidth, low latency, and short round-trip times (RTTs).

Traditional TCP performs poorly in such settings because it reacts too strongly to packet loss, causing high delay and inefficient utilization.

DCTCP solves this by using Explicit Congestion Notification (ECN) for fine-grained congestion feedback, allowing the sender to adjust its sending rate proportionally to the level of congestion instead of reacting to packet drops.

Core Principles

1. Early Congestion Detection

Network switches mark packets with the Congestion Experienced (CE) codepoint once the queue length exceeds a predefined threshold K .

This marking lets senders detect congestion before packet loss occurs, avoiding large queue buildup.

2. Proportional Reaction to Congestion

Instead of cutting the congestion window in half after every congestion signal (as in TCP Reno), DCTCP reacts proportionally.

The sender measures the fraction of marked packets (α) during one RTT:

$$\alpha \leftarrow (1 - g) \times \alpha + g \times F$$

where F is the fraction of ECN-marked packets in the last window and g ($0 < g < 1$) is the weight given to new samples.

- If $\alpha \approx 0 \rightarrow$ mild congestion $\rightarrow$ minor reduction in cwnd.
- If $\alpha \approx 1 \rightarrow$ heavy congestion $\rightarrow$ cwnd halved (like regular TCP).

3. Multi-Bit Feedback from ECN

DCTCP effectively extracts multi-level feedback from ECN's single-bit mechanism. The receiver echoes back each marked packet using the ECN-Echo flag in ACKs, allowing the sender to compute α precisely.

4. Compatibility with Standard TCP

DCTCP retains most standard TCP mechanisms — slow start, additive increase, and retransmission recovery — differing only in how it interprets ECN signals.

This makes it easy to deploy in existing TCP/IP stacks.

Algorithm Summary

- At the Switch:
Mark packets when queue occupancy $> K$ (marking threshold).
- At the Receiver:
Set ECN-Echo bits in ACKs to inform the sender of received marks until a Congestion Window Reduced (CWR) flag is acknowledged.
- At the Sender:
Maintain and update α (fraction of marked packets) every RTT.
Adjust congestion window $cwnd$ according to α :

$$cwnd \leftarrow cwnd \times (1 - \alpha/2)$$

This ensures smooth adaptation: smaller reductions for light congestion, stronger ones for heavier congestion.

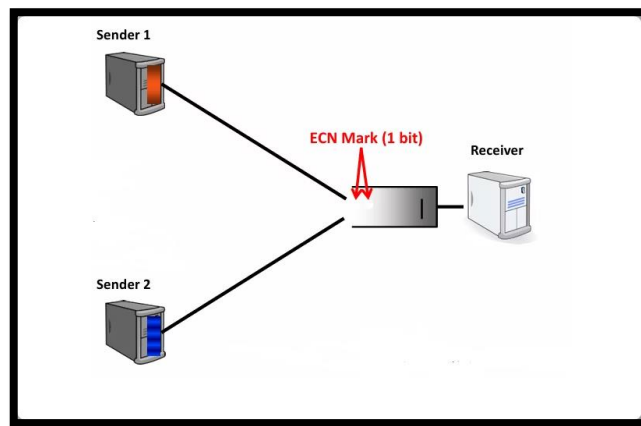
Key Insights

- α approximates the probability that the queue length exceeds threshold K .
- DCTCP gently reduces $cwnd$ as soon as congestion starts, keeping queues short while maintaining high throughput.
- When congestion is severe ($\alpha = 1$), it behaves like standard TCP, cutting $cwnd$ by 50%.

Conclusion

DCTCP achieves high throughput with low latency by providing proportional, ECN-based congestion control.

It maintains small queue occupancies and avoids packet loss — making it ideal for data centers running latency-sensitive applications such as cloud storage, distributed computing, and real-time analytics.



Fairness Metrics

Jain's Index:

- Formula:

$$J(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \cdot \sum_{i=1}^n x_i^2}$$

where x_i is the throughput (or resource share) of flow i , and n is the number of flows.

- Range:

$$\frac{1}{n} \leq J \leq 1$$

$1 =$ perfectly fair (all flows get the same rate).

Closer to $\frac{1}{n}$ = very unfair (one flow dominates).

Use case: Popular in TCP fairness evaluation because it's simple, normalized, and widely understood.

G Index:

- Formula (one version):

$$G = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n^2 \bar{x}}$$

where $\bar{x}$ is the mean throughput.

- Range:

$$0 \leq G \leq 1$$

0 = *perfectly fair (everyone equal)*.

1 = *maximum inequality (one gets everything)*.

- Interpretation:
 - Based on inequality measurement (borrowed from economics).
 - A higher G means more disparity between flows.

Use case: Sometimes preferred when you want a sensitive measure of inequality, beyond Jain's index.

Max-Min Fairness:

Definition: A resource allocation is *max-min fair* if no flow can increase its allocation without decreasing the allocation of another flow that already has an equal or smaller share.

An allocation $x = (x_1, \dots, x_n)$ is said to be max-min fair if it satisfies:

No allocation x_i can be increased without decreasing some $x_j \leq x_i$.

Formally, a vector of allocations is max-min fair if for any feasible vector y ,

$$y_i > x_i \implies \exists j \text{ such that } y_j < x_j \leq x_i.$$

Characteristics:

- Prioritizes flows with smaller allocations.
- Ensures that the minimum allocation is maximized subject to capacity constraints.
- Provides a deterministic allocation rule, often used in bandwidth-sharing models.

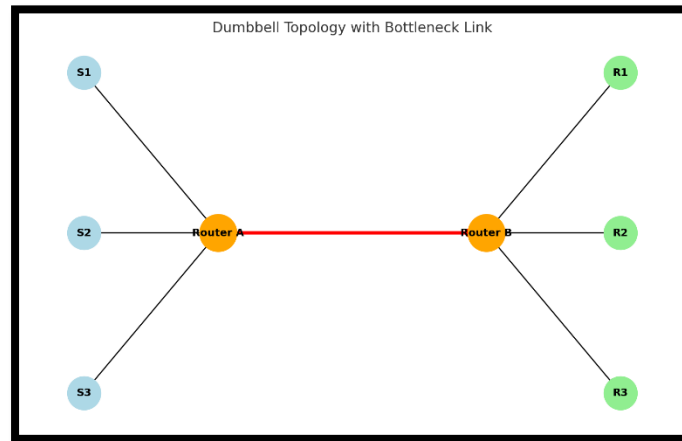
Use case: Used in theoretical fairness frameworks, often in resource allocation algorithms (not just TCP).

Dumbbell topology & Bottleneck:

A dumbbell topology is a standard experimental setup for evaluating congestion control algorithms. In this topology, multiple senders connect through a router to a single shared link, which then connects to another router and multiple receivers. The central shared link resembles the “handle” of a dumbbell, while the senders and receivers form the “weights.”

The bottleneck refers to this shared link between the routers. Since all flows must pass through it, its limited capacity compared to the aggregate sending rates makes it the primary point of congestion. When the bottleneck link is overloaded, queues build up and packets may be delayed or dropped.

Studying protocol behavior at the bottleneck is essential for understanding throughput, latency, and fairness. For this reason, the dumbbell topology is one of the most widely used network models in congestion control research.



Experimental Setup

Experiment Design

To evaluate the behavior of different TCP congestion control protocols, a series of simulations were conducted using the NS-3 network simulator.

The experiments were divided into two main environment types, each designed to analyze different aspects of protocol performance under controlled network conditions.

Environment Type 1 – Single Protocol

In this setup, both flows in the simulation used the same TCP congestion control protocol (Reno, Cubic, BBR, or DCTCP).

Two types of experiments were performed under this environment:

1. Same RTT for Both Flows:

- Both flows were assigned the same RTT value.
- The RTT was varied uniformly across simulation runs: 10, 20, 40, 60, 100, 200, 300, 400, and 500 ms.
- This setup allows observing how a single protocol behaves under different delay conditions while keeping both flows symmetric.

2. Different RTTs for Each Flow:

- Flow 1 was fixed at an RTT of 100 ms.
- Flow 2 was assigned an RTT scaled relative to Flow 1: $0.1 \times \text{RTT}_1$, $0.2 \times \text{RTT}_1$, $0.4 \times \text{RTT}_1$, $0.6 \times \text{RTT}_1$, $0.8 \times \text{RTT}_1$, RTT_1 , $1.2 \times \text{RTT}_1$, $1.4 \times \text{RTT}_1$, $1.6 \times \text{RTT}_1$, $1.8 \times \text{RTT}_1$, and $2 \times \text{RTT}_1$.
- This variation helps analyze how the same protocol handles RTT asymmetry between two competing flows.

Environment Type 2 – Two Protocols (Mixed Senders)

In this setup, the simulation included two flows, each using a different TCP congestion control protocol.

This environment was designed to study fairness and interaction between protocols under identical network conditions.

1. Same RTT for Both Flows:

- Both flows were assigned the same RTT value, varied uniformly across simulations (10–500 ms).
- This experiment shows how two different protocols compete for bandwidth when delay conditions are identical.

2. Different RTTs for Each Flow:

- Flow 1 had a fixed RTT of 100 ms, while Flow 2's RTT varied as a fraction or multiple of Flow 1's RTT (from $0.1 \times \text{RTT}_1$ up to $2 \times \text{RTT}_1$).
- This setup examines how RTT asymmetry affects fairness between different congestion control algorithms.

Flow Count

All simulations were performed using two concurrent flows.

During preliminary tests, it was observed that increasing the number of flows beyond two made the analysis significantly more complex and less clear without adding substantial new insights.

Therefore, focusing on two flows provided a balance between simplicity and the ability to clearly evaluate fairness and performance across protocols.

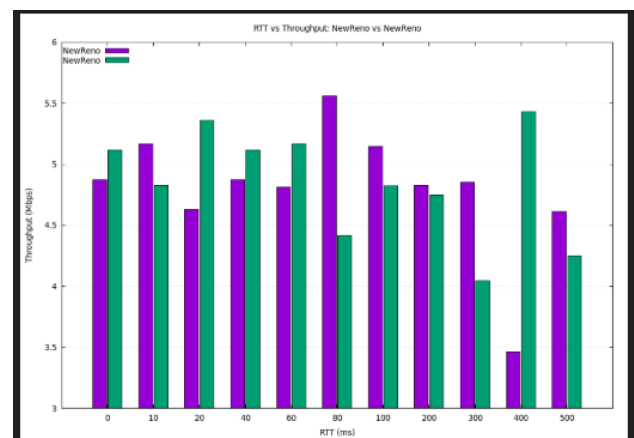
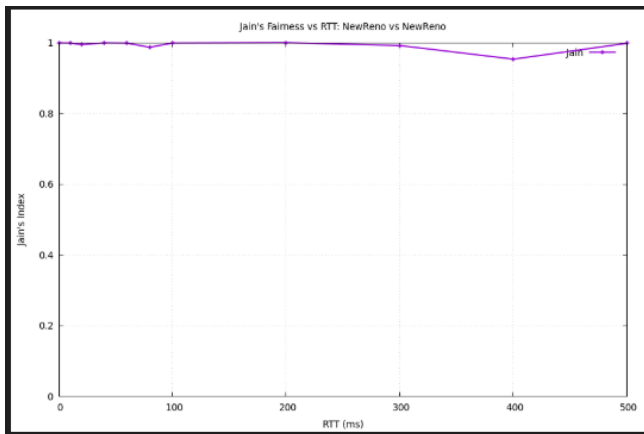
Results and Analysis

Single-Protocol Results (New Reno, CUBIC, DCTCP, BBR)

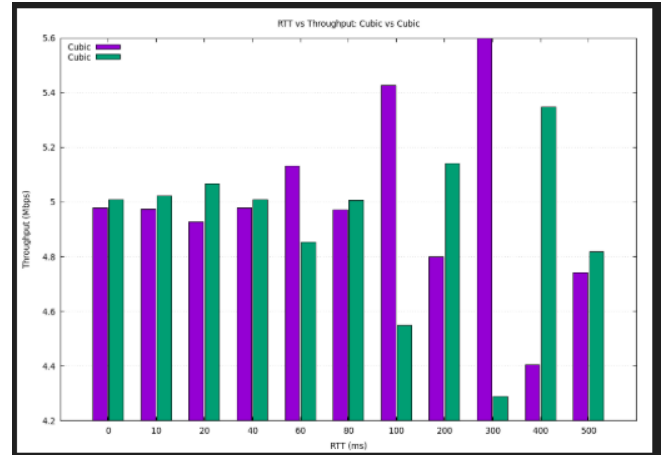
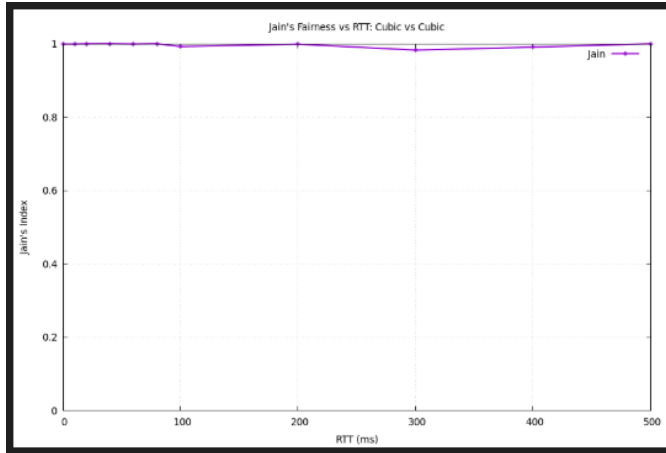
With the same RTT value:

When both flows use the same congestion control protocol and operate with identical RTT values, the fairness between them remains stable regardless of the actual RTT value used. In other words, if both flows experience the same delay (for example, 10 ms, 100 ms, or 500 ms), they share the bottleneck link evenly, and Jain's fairness index stays constant and close to 1. This indicates that fairness depends on the relative difference in RTTs, not on their absolute magnitude. Increasing the RTT for both flows by the same amount only reduces the total throughput due to longer feedback loops but does not affect how fairly the bandwidth is divided between them.

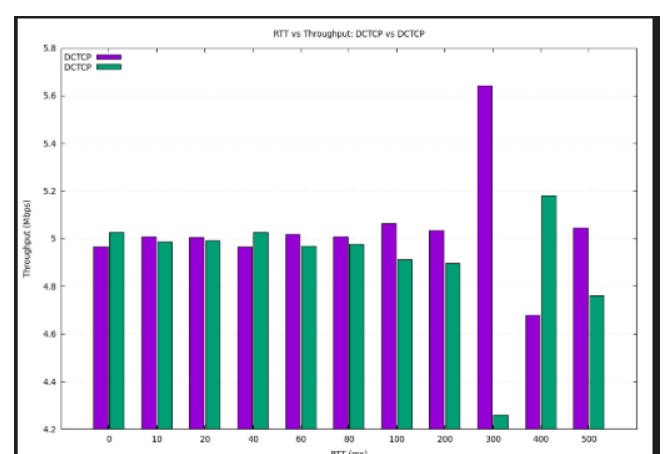
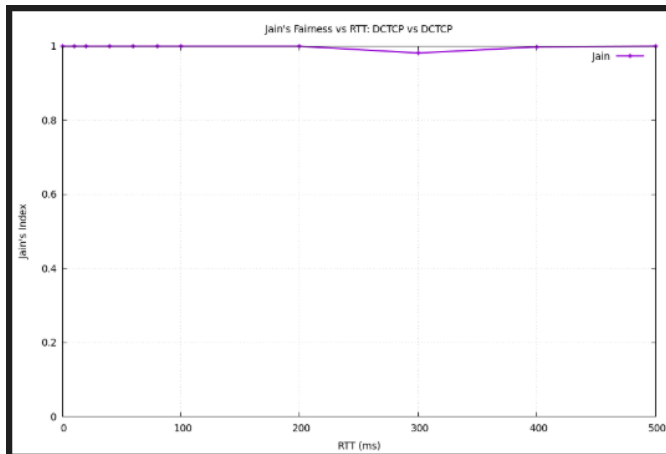
1. NewReno



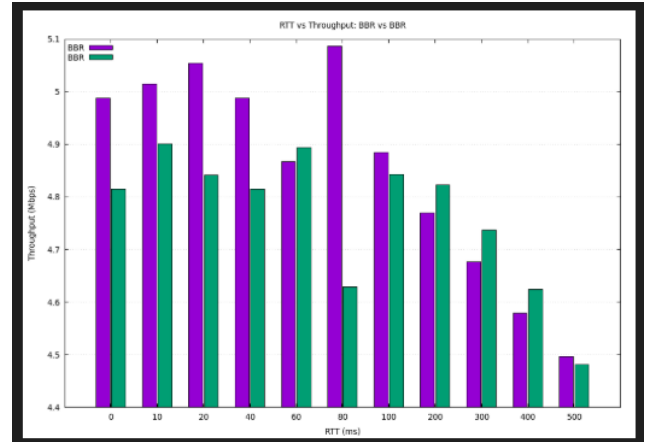
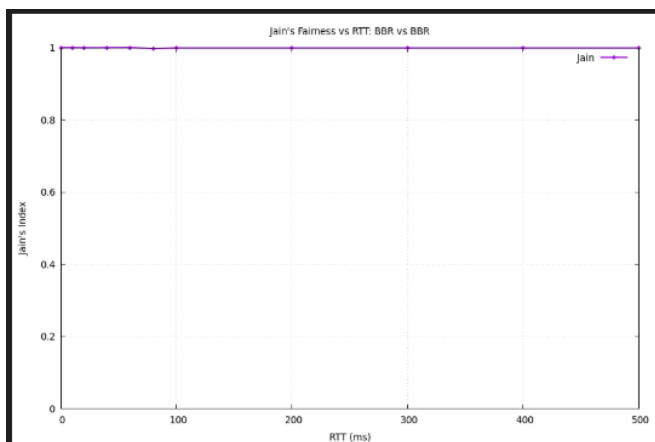
2. CUBIC



3. DCTCP



4. BBR



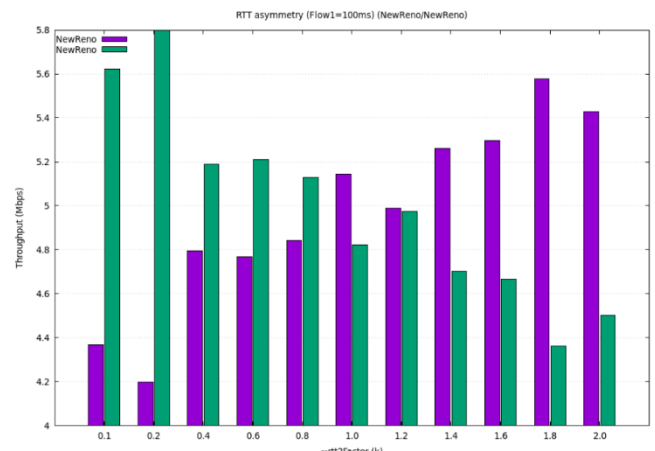
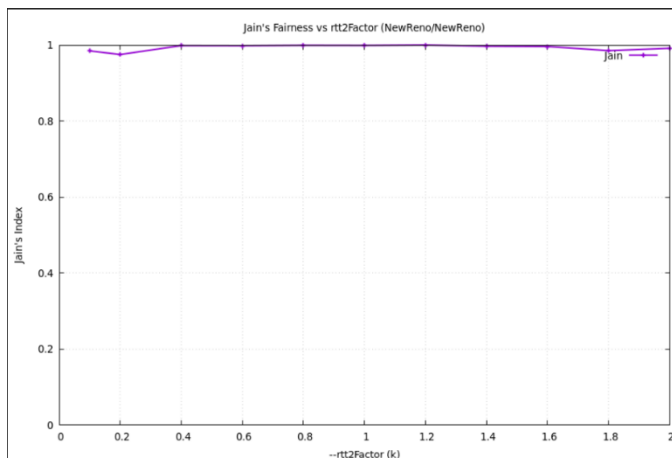
With different RTT values:

1. NewReno

The flow with the smaller RTT consistently achieves a higher throughput than the flow with the larger RTT. The fairness is almost perfect only when the RTTs are equal ($k=1.0$) or very close.

Interpretation: With a much smaller RTT, Flow 2 receives its ACKs much faster. This allows its Congestion Window (cwnd) to grow more rapidly according to the Additive-Increase Multiplicative-Decrease (AIMD) algorithm of NewReno. It can "claim" spare bandwidth on the link more quickly after a packet loss event, starving the slower-RTT flow.

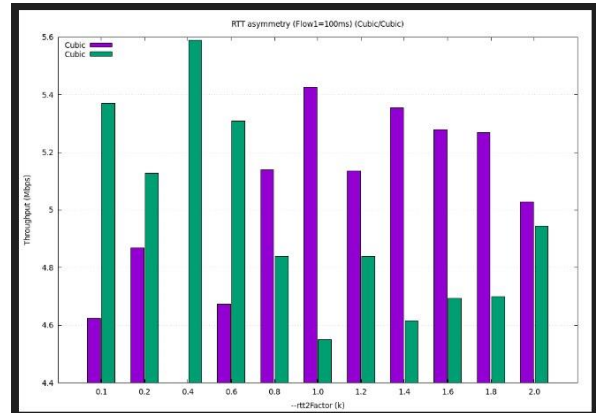
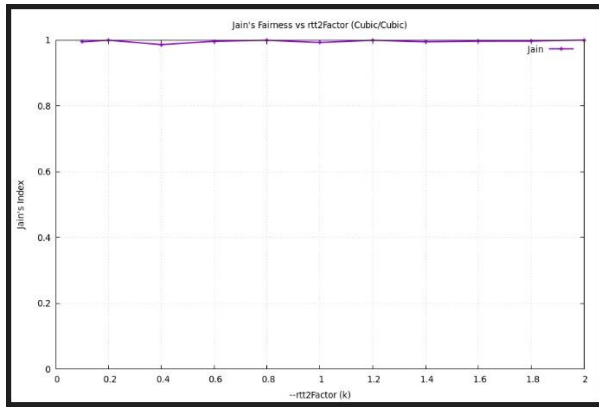
$k=1.2$ (Flow 2 RTT=120ms): Interestingly, this point is also very fair (4.99 vs 4.98 Mbps). This suggests the unfairness only becomes significant with larger RTT disparities.



2. CUBIC

When two Cubic flows share a bottleneck, the flow with the smaller RTT gains a throughput advantage. In this experiment (Flow 1 = 100 ms, Flow 2 = 10 – 200 ms), throughput symmetry occurs only near equal RTTs. Jain's index remains close to 1,

showing moderate fairness overall, but the consistent advantage of the low-RTT flow reveals Cubic's intrinsic RTT bias.

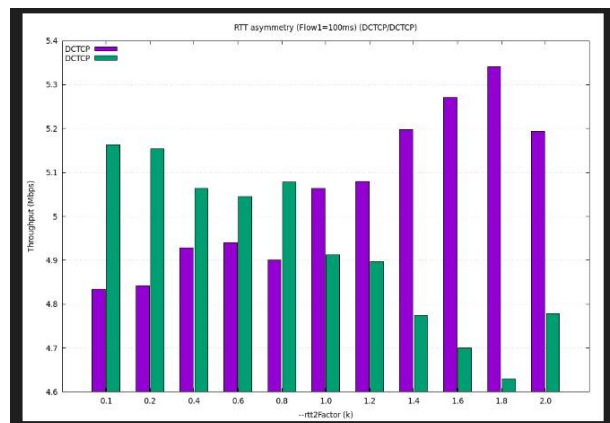
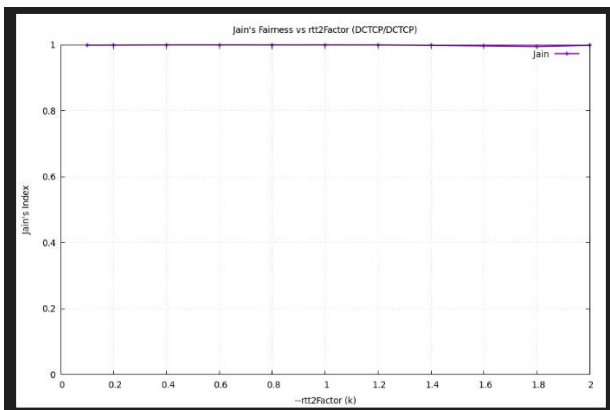


3. DCTCP

When both flows use DCTCP, fairness remains nearly perfect across RTT asymmetry ranging from 10 ms to 200 ms (Flow 1 fixed at 100 ms). Throughput differences between the two flows are negligible, and Jain's index stays at 1.0. This confirms that DCTCP's ECN-based mechanism provides RTT-independent congestion control, in contrast to Cubic's RTT bias where lower-RTT flows dominate.

However, we can observe that as the RTT increases, the throughput slightly decreases, since the flow with the higher RTT experiences longer feedback delays — by the time its acknowledgments return, the queue is already filled by the other flow, reducing its sending opportunity.

DCTCP thus achieves perfect fairness when ECN marking is symmetric, but higher RTTs still slightly limit throughput due to delayed congestion feedback.

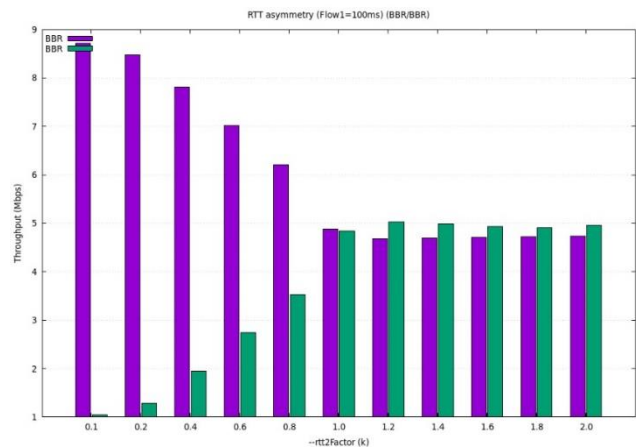
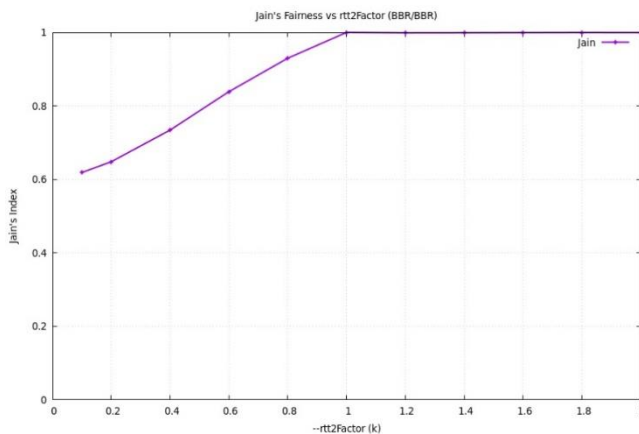


4. BBR

In this experiment, both flows use the BBR protocol. The first flow has a fixed RTT of 100 ms, while the second flow's RTT increases gradually. As the RTT of the second flow grows, its throughput increases until both flows reach a similar rate. This occurs because BBR adjusts its congestion window based on measured RTT and bottleneck bandwidth.

When RTT increases, BBR enlarges its window to maintain full link utilization, allowing the higher-RTT flow to catch up. Eventually, both flows achieve nearly equal throughput, showing that BBR maintains good fairness between flows even under RTT asymmetry. Note that, according to the graph, the unfairness appears mainly when the smaller RTT is around 100 ms.

When $k > 1$, one flow still has a smaller RTT, but the throughput gap becomes smaller, meaning BBR maintains a relatively balanced performance even with RTT differences.

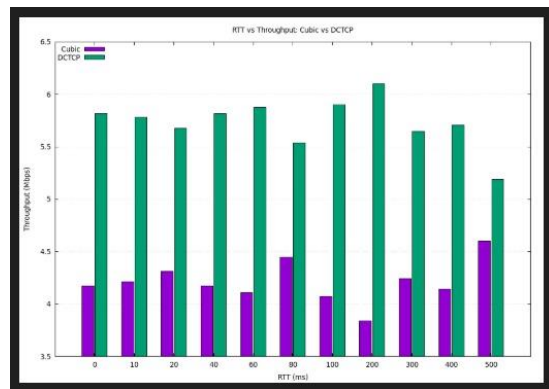
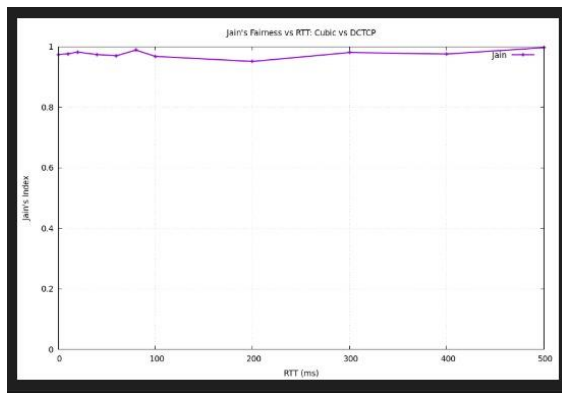


Mixed-Protocol Fairness (Protocol A vs Protocol B)

With the same RTT value:

1. CUBIC & DCTCP

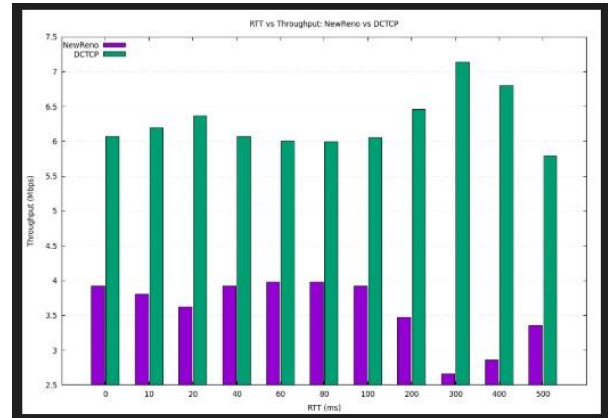
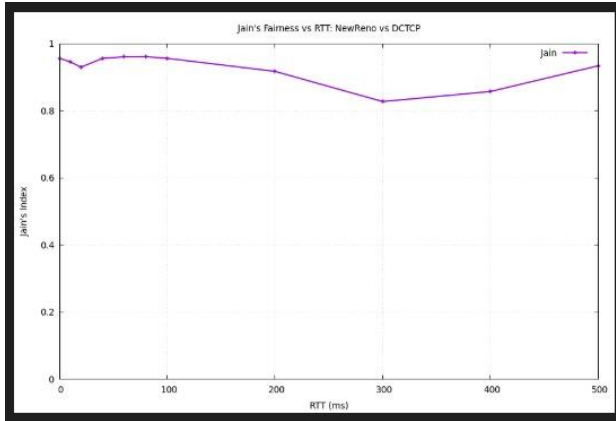
When Cubic and DCTCP share a link under increasing RTT conditions, DCTCP consistently outperforms Cubic, maintaining significantly higher throughput while Cubic's performance remains limited by its loss-based control. Jain's fairness index stays near 1, suggesting stable coexistence but clear imbalance. DCTCP's ECN-based congestion control provides superior efficiency and lower RTT dependence, highlighting its advantage over traditional loss-triggered algorithms such as Cubic.



2. NewReno & DCTCP

DCTCP maintains a consistently high throughput, mostly between 6 and 7 Mbps, regardless of the RTT. It shows only a slight decline as RTT becomes very large. NewReno is consistently and significantly lower. With high RTT, this window growth is extremely slow. After a packet loss, it takes a long time to recover. DCTCP's constant pressure on the bottleneck prevents NewReno from ever getting a foothold, leading to its near-complete starvation at high RTT.

Total Efficiency Drops: The sum of both throughputs (the height of the two lines combined) decreases as RTT increases. This means the bottleneck link is being significantly underutilized, especially at high RTTs.

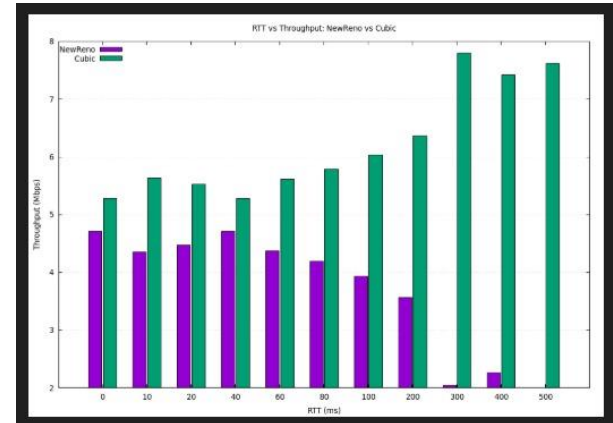
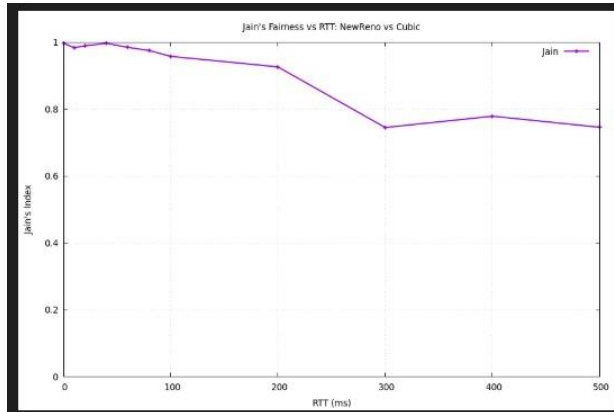


3. NewReno & CUBIC

CUBIC maintains a high and very stable throughput, consistently between 6.5 and 7 Mbps, across all RTT values. It is almost completely unaffected by the increase in latency.

NewReno is significantly lower. It starts around 4.5 Mbps and its throughput steadily decreases as the RTT increases. CUBIC: Its performance is resilient to RTT due to its aggressive cubic growth function making it much less dependent on a fast ACK clock than NewReno.

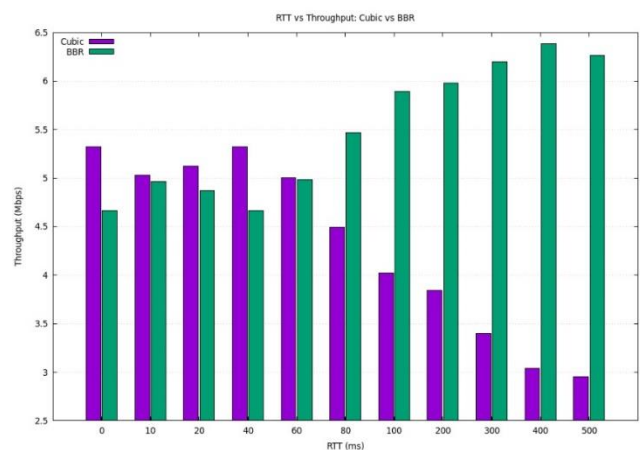
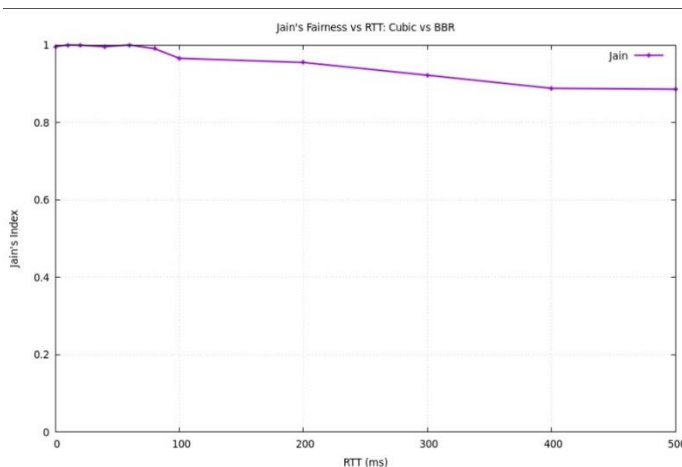
NewReno: As a standard loss-based algorithm, it suffers from high RTT. Its linear window growth is slow, and recovery from packet loss is sluggish. CUBIC's constant aggression prevents NewReno from ever achieving a fair share.



4. CUBIC & BBR

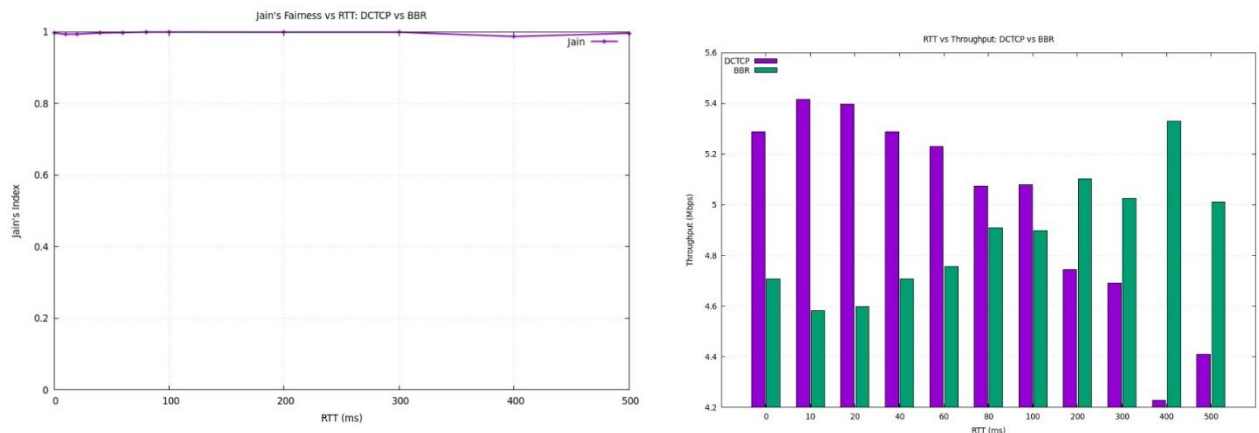
In this experiment, both CUBIC and BBR experience the same RTT, which increases gradually.

As RTT grows, BBR maintains or even improves its throughput, while CUBIC's performance drops significantly. This is because CUBIC, a loss-based protocol, reacts to higher RTT by reducing its sending rate interpreting the delay as a sign of congestion. BBR, on the other hand, estimates the bottleneck bandwidth and adjusts its congestion window to keep the link fully utilized, regardless of RTT. As a result, BBR demonstrates much higher stability and throughput under increasing latency compared to CUBIC.



5. DCTCP & BBR

In this experiment, both CUBIC and BBR experience the same RTT, which increases gradually. As RTT grows, BBR maintains or even improves its throughput, while CUBIC's performance drops significantly. This is because CUBIC, a loss-based protocol, reacts to higher RTT by reducing its sending rate — interpreting the delay as a sign of congestion. BBR, on the other hand, estimates the bottleneck bandwidth and adjusts its congestion window to keep the link fully utilized, regardless of RTT. As a result, BBR demonstrates much higher stability and throughput under increasing latency compared to CUBIC.

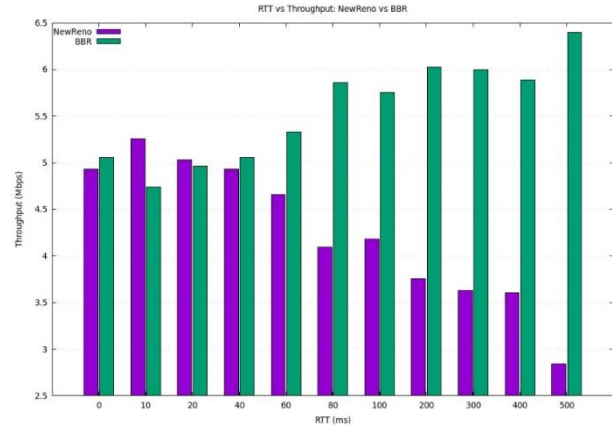
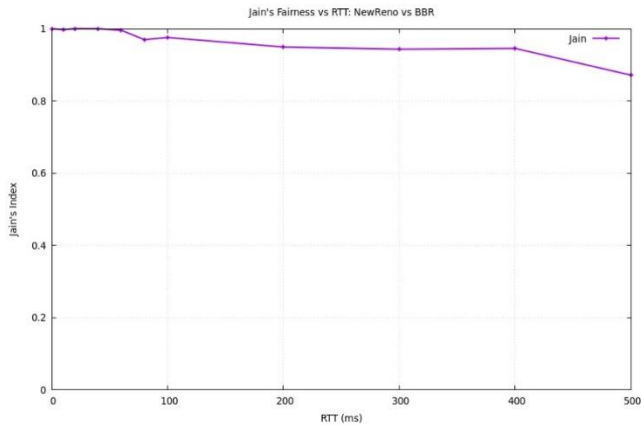


6. BBR & NewReno

In this experiment, both BBR and NewReno experience the same RTT, which gradually increases.

As RTT grows, BBR maintains or even improves its throughput, while NewReno's performance consistently decreases. This happens because BBR estimates the available bandwidth and adjusts its sending window to fully utilize the link, regardless of the delay.

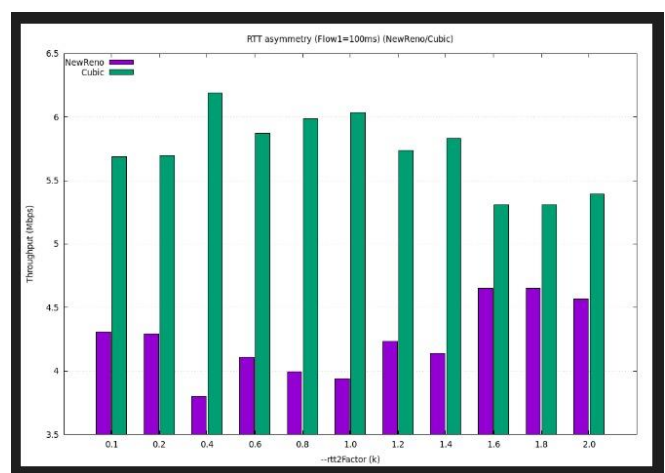
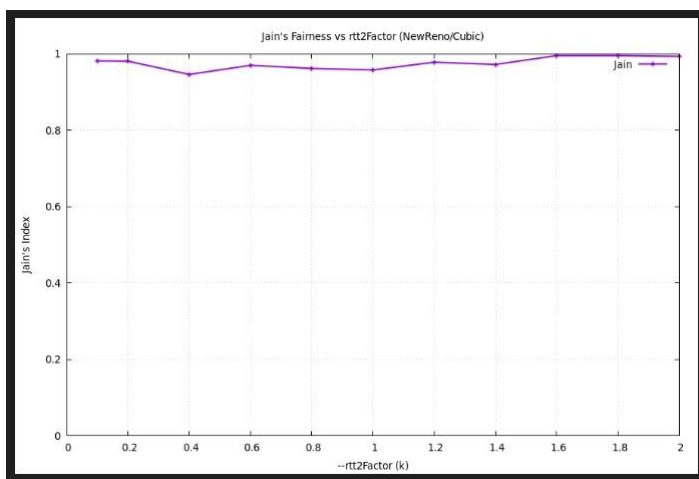
In contrast, NewReno — being loss-based — reduces its sending rate as RTT increases, since it interprets higher delay as congestion. Therefore, BBR shows greater stability and efficiency under high-latency conditions compared to NewReno.



With different RTT values:

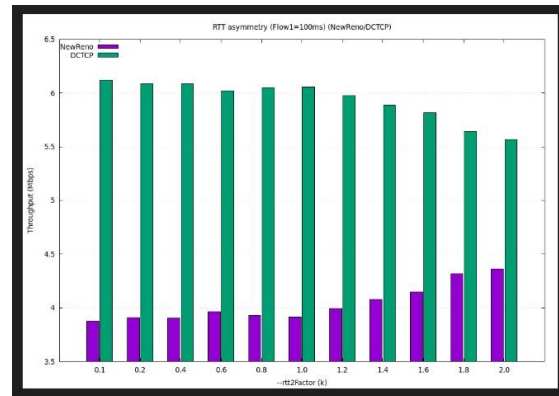
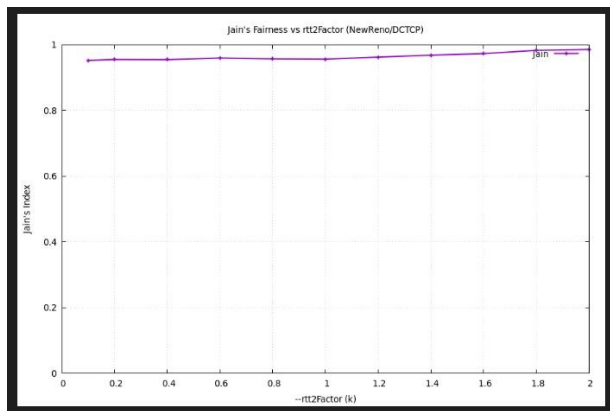
1. NewReno & CUBIC

When NewReno and Cubic share a bottleneck, the Cubic flow consistently dominates regardless of RTT asymmetry. Cubic's faster cubic-shaped congestion window growth allows it to utilize more bandwidth, while NewReno's linear AIMD mechanism limits its recovery speed. Consequently, the throughput ratio remains skewed in Cubic's favor, and Jain's fairness index drops below 1, indicating persistent unfairness between the two protocols.



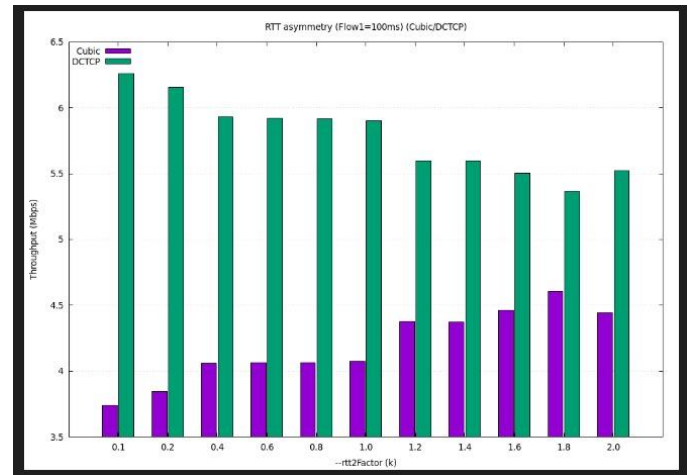
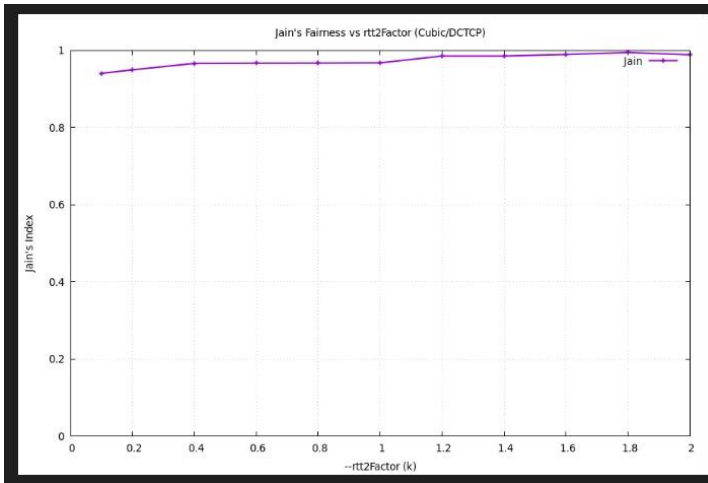
2. NewReno & DCTCP

When NewReno and DCTCP share a bottleneck, DCTCP consistently outperforms NewReno, obtaining significantly higher throughput across all RTT asymmetries. This stems from DCTCP's ECN-based fine-grained congestion control, which maintains high utilization and avoids packet loss, while NewReno's loss-based AIMD mechanism reduces its rate sharply after congestion events. Jain's fairness index (~ 0.9) reflects this imbalance, confirming that the two protocols coexist poorly — DCTCP dominates and NewReno is strongly penalized.



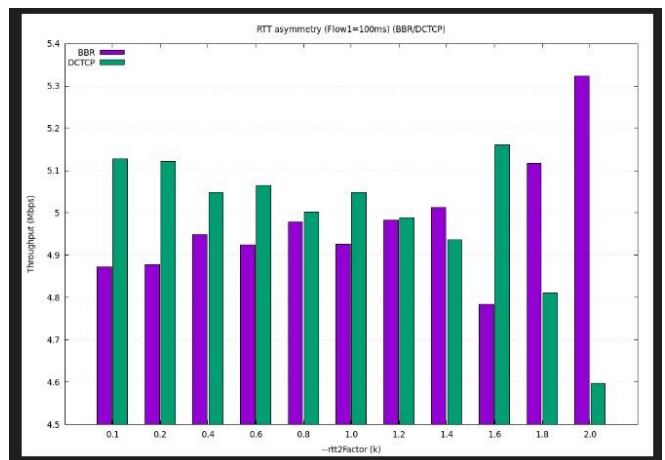
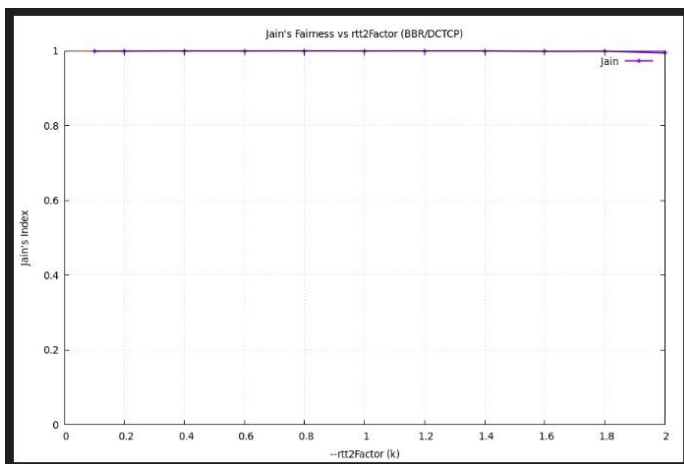
3. CUBIC & DCTCP

When Cubic and DCTCP share a bottleneck, DCTCP clearly dominates regardless of RTT asymmetry. Its ECN-driven control reacts more quickly and smoothly to congestion, maintaining higher throughput with minimal loss, while Cubic's loss-based behavior leads to frequent reductions and slower recovery. As a result, fairness is limited ($Jain \approx 0.9$) and Cubic consistently receives less bandwidth. This demonstrates that ECN-based algorithms like DCTCP can significantly outperform traditional loss-based TCP variants when coexisting on the same link.



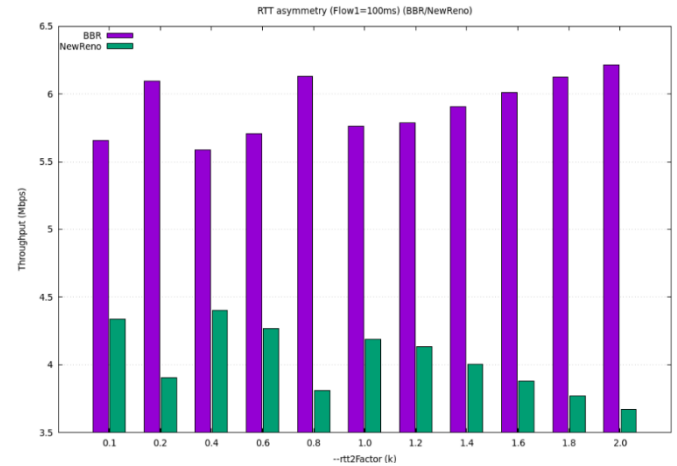
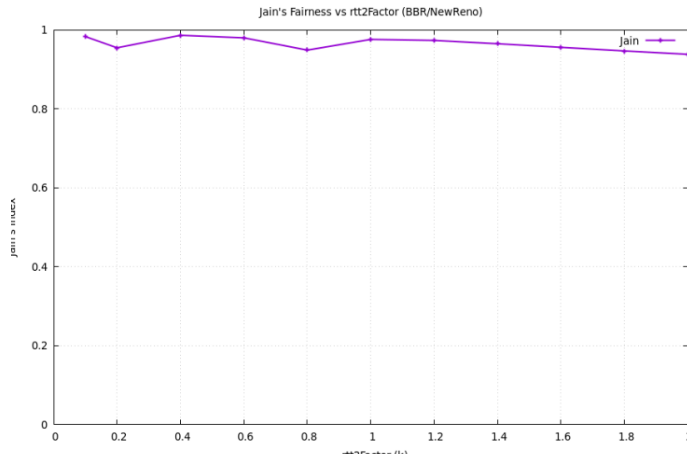
4. BBR & DCTCP

When BBR and DCTCP share the same bottleneck, both flows maintain nearly equal throughput across all RTT asymmetries. The interaction between BBR's model-based pacing and DCTCP's ECN-based feedback leads to smooth congestion management with minimal RTT dependence. Jain's fairness index remains at 1, confirming ideal fairness and coexistence. This shows that modern algorithms like BBR and DCTCP can efficiently share the network while achieving stable and fair throughput distribution.



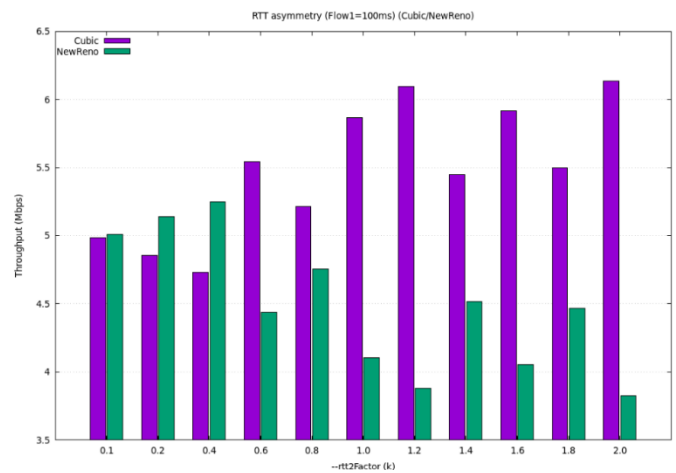
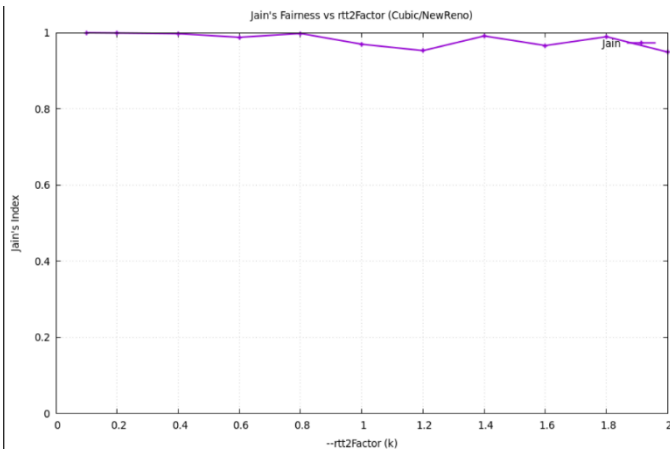
5. BBR & NewReno

BBR seizes the vast majority of the bandwidth, and NewReno is reduced to scavenging the remnants, with RTT playing only a minor role in determining the degree of this starvation.



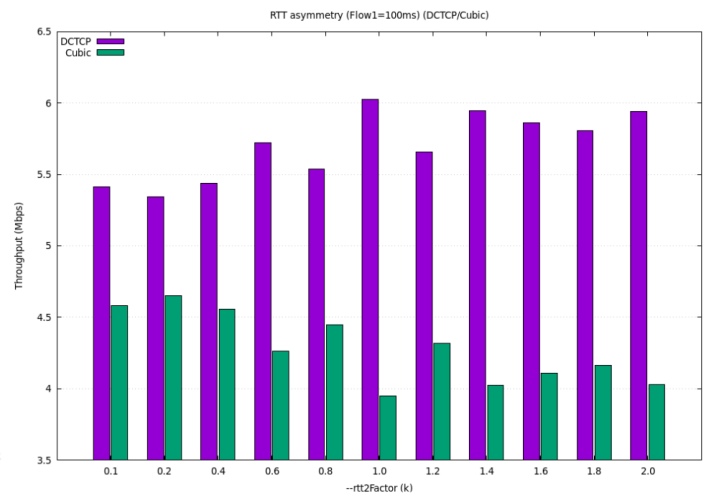
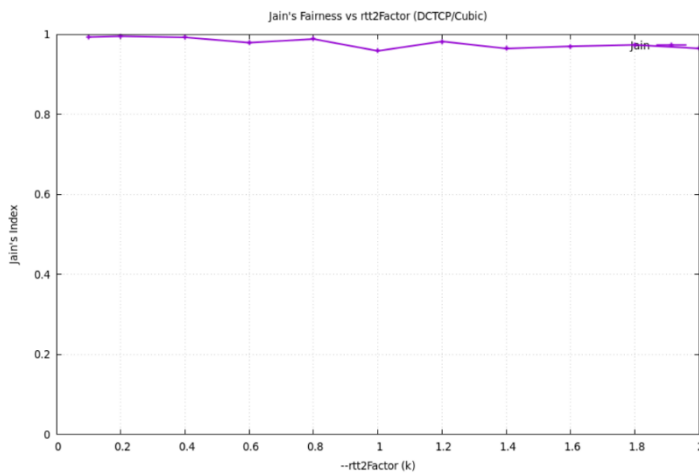
6. CUBIC & NewReno

When RTTs are similar CUBIC's aggressive algorithm gives it a clear advantage, same when New Reno has bigger RTT. When NewReno has an RTT advantage The competition is much fairer. A significantly smaller RTT allows NewReno to compensate for its less aggressive algorithm. Its faster ACK clock enables it to increase its window more rapidly, allowing it to nearly match CUBIC's throughput



7. DCTCP & CUBIC

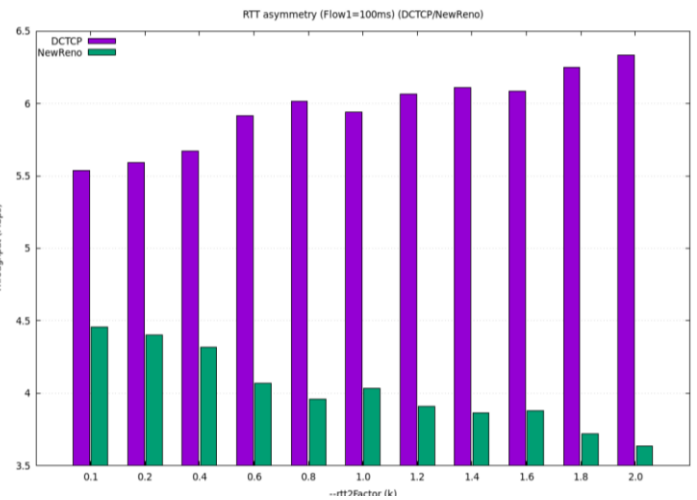
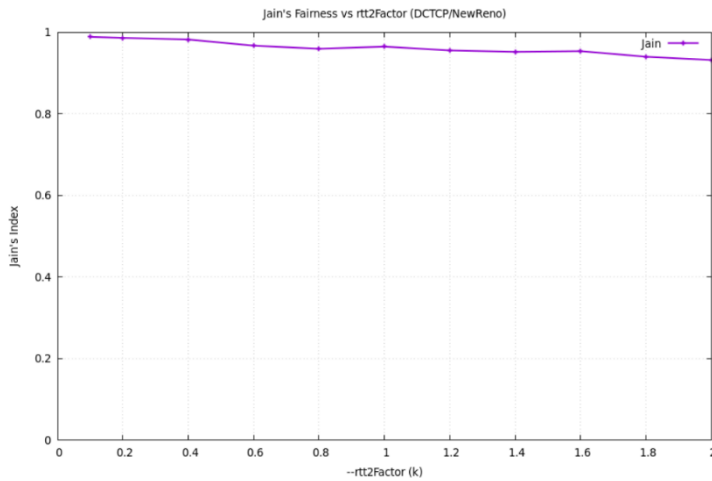
In a mixed CCA environment, DCTCP generally dominates CUBIC due to its more efficient ECN-based mechanism. However, CUBIC's aggressive design makes it a far more formidable competitor than NewReno. The final outcome is a tug-of-war between CUBIC's ability to exploit RTT advantages and DCTCP's inherent algorithmic superiority. According to the graph dtccp dominates although when cubic has smaller rtt it puts a fight but decreases with higher rtt.



8. DCTCP & NewReno

The result is that DCTCP quickly fills the bottleneck link and keeps it full, while NewReno, constantly backing off drastically due to packet loss, never gets a chance to

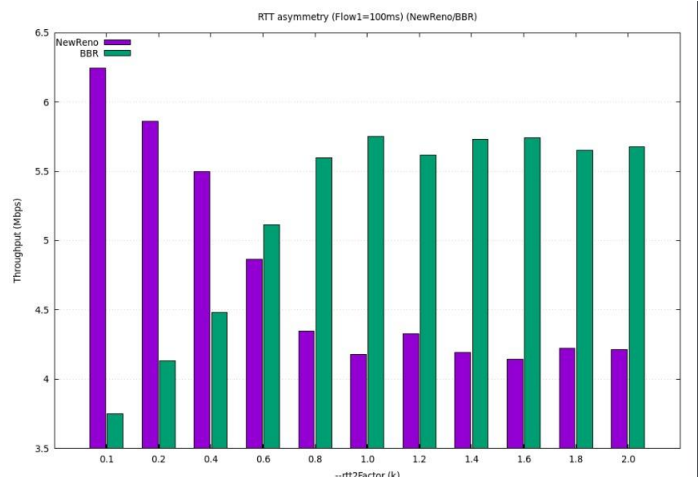
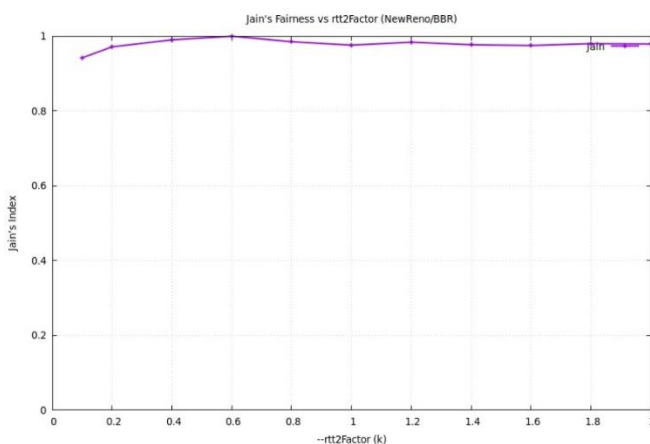
claim its fair share. The results are unrelated to the rtt but with of how the protocols behave



9. NewReno & BBR

As BBR's RTT increases, it expands its congestion window to keep the network fully utilized. This allows BBR to send more data and achieve higher throughput.

However, because both flows share the same bottleneck, BBR's increased sending rate reduces the available bandwidth for NewReno, whose RTT remains constant and reacts to congestion by decreasing its sending rate. As a result, BBR's throughput increases at the expense of NewReno's performance showing that BBR can dominate loss-based protocols when competing for the same link.



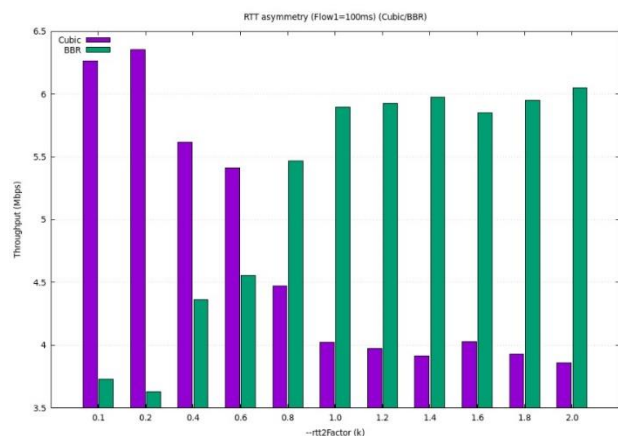
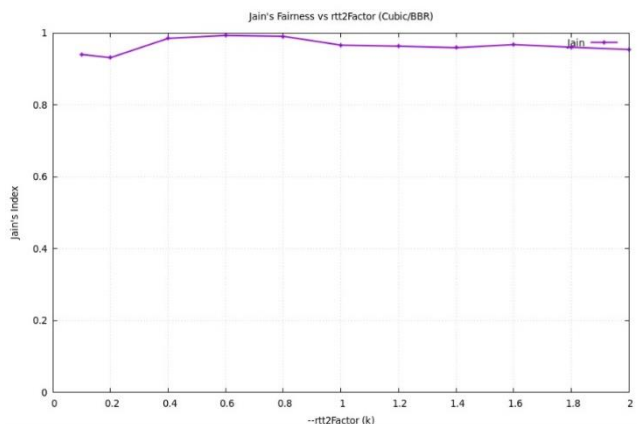
10. CUBIC & BBR

When competing with CUBIC, BBR's throughput increases as its RTT grows, while CUBIC's performance declines.

This happens because CUBIC, being a loss-based protocol, interprets higher RTT or queue buildup as a sign of congestion and reduces its sending rate.

In contrast, BBR increases its congestion window with higher RTT to keep the network pipe full and maintain high utilization.

As a result, BBR dominates the shared bottleneck and achieves higher throughput at the expense of CUBIC's rate, demonstrating BBR's aggressive behavior in mixed-protocol environments.



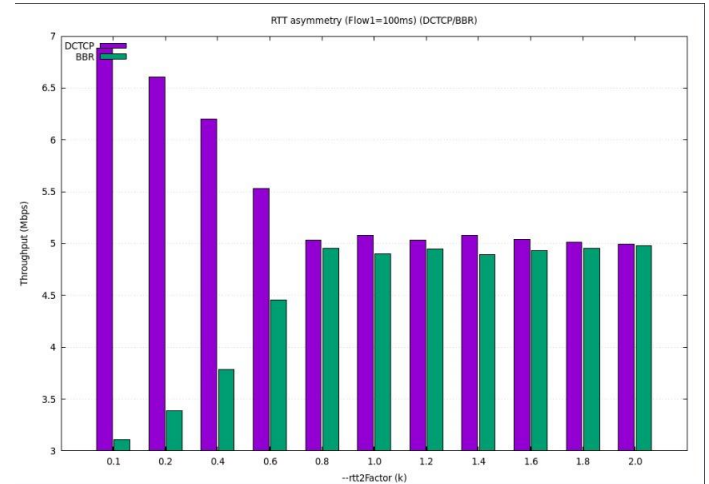
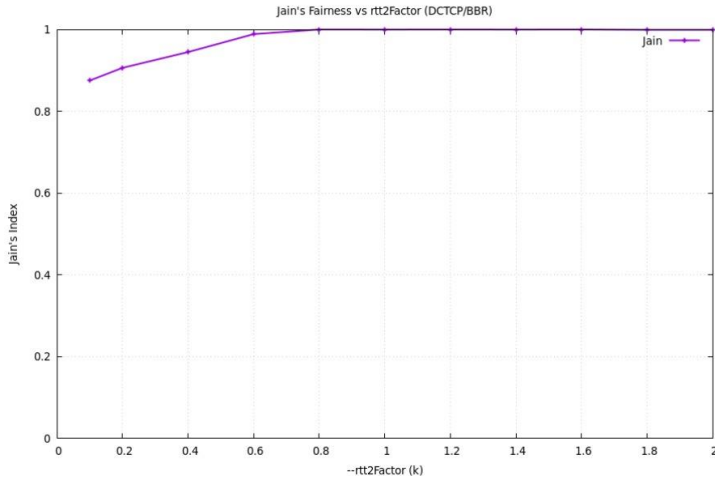
11. DCTCP & BBR

When competing with DCTCP, BBR initially shows lower throughput since DCTCP quickly reacts to congestion and maintains efficient queue control.

However, as BBR's RTT increases, it expands its congestion window to sustain high utilization, while DCTCP slightly reduces its rate due to ECN feedback.

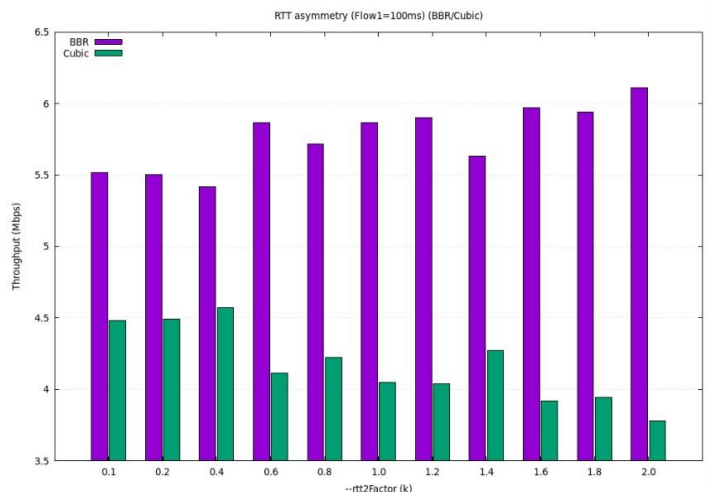
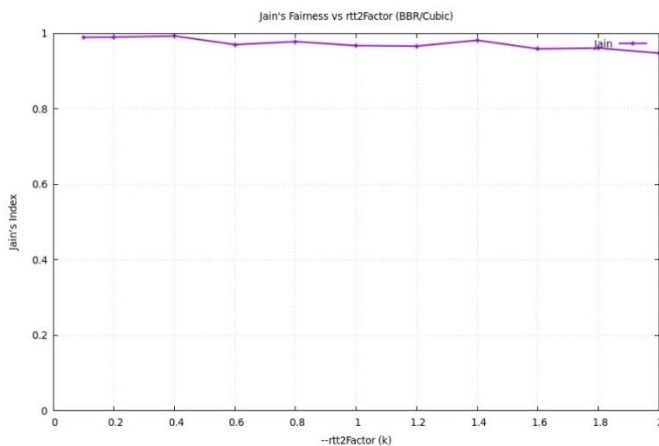
Eventually, both protocols reach a point of balance where their throughputs become nearly equal, indicating that BBR adapts to network conditions until it matches DCTCP's performance.

This demonstrates that while DCTCP is more responsive to congestion, BBR gradually compensates by increasing its sending rate as RTT grows.



12. BBR & CUBIC

In this experiment, CUBIC's RTT varies while BBR's RTT remains constant at 100 ms. As CUBIC's RTT increases, its throughput significantly decreases because CUBIC is a loss-based protocol that reacts to longer RTT and congestion by reducing its sending rate. Meanwhile, BBR maintains a stable throughput since it estimates the available bandwidth rather than relying on packet loss or RTT variations. As a result, BBR remains dominant across all RTT values, clearly outperforming CUBIC and demonstrating its robustness against delay sensitivity.



Conclusions and Future Work

- When Dctcp is paired with loss based CCA's -NewReno, Cubic- it captures the bandwidth unrelated to RTT but due to its advanced algorithm dealing with congestion.
- NewReno gets much less throughput when paired with other CCA's so it nearly losses bandwidth every time.
- Cubic manages to get decent throughput when paired with other CCA's due to its fast growing cwnd.
- when BBR has similar RTT to Dctcp and itself it's protocol friendly. It manages to take appropriate throughput without outperforming the other protocol. But' with NewReno and Cubic it causes outperformances.
- The Max-Min Fairness principle holds when both protocols are the same and both flows have the same RTT. In other cases, it is difficult to determine, since both flows attempt to fully utilize the available bandwidth, but in practice they do not receive an equal share of it.
- Modern protocols such as BBR and DCTCP are capable of sharing bandwidth efficiently and fairly, even under complex network conditions.

In the future, this research can be expanded by increasing the number of concurrent flows, for example to three or four, to analyze how fairness and throughput are affected under higher network load. We can also examine additional parameters such as packet size and flow start time to gain a deeper understanding of protocol behavior in more realistic datacenter conditions. While fairness in this work was measured using Jain's Fairness Index, future studies can also use the G-index for a broader and more detailed evaluation of fairness. Studying these factors will provide a deeper understanding of how different congestion control protocols behave under more complex and realistic datacenter conditions.

Objectives and Project Plan

Goals

- Gain a solid understanding of TCP congestion control and its main variants (such as Reno, Cubic, and DCTCP).
- Ramp up and become proficient with the NS-3 simulator.
- Build and configure a dumbbell topology in NS-3 for controlled experiments.
- Simulate and test multiple TCP congestion control protocols on the dumbbell topology.
- Evaluate and analyze fairness between TCP connections using different congestion control protocols.
- Present results by creating graphs and plots (e.g., throughput vs. number of connections) to compare protocol performance.
- Document the results clearly and summarize the findings.

Estimated Timetable

Week 1:

- Introduction to the project.
- Define initial objectives and scope.

Weeks 2–3: Research and Learning

- Study the basics of congestion control in TCP.
- Review different TCP congestion control protocols (e.g., Reno, Cubic, DCTCP).
- Read key research articles and background material.
- Get familiar with the NS-3 simulator through tutorials and example scripts.

Weeks 4–5: Initial Experiments

- Set up and run experiments on the dumbbell topology in NS-3.
- Test TCP connections using different congestion control algorithms.
- Begin defining metrics for evaluating performance (e.g., throughput, fairness).

Weeks 6–7: Fairness Analysis

- Implement experiments to compare TCP congestion control protocols under the dumbbell topology.
- Collect performance data (throughput, latency, packet loss).
- Start analyzing fairness across different protocols.

Weeks 8–10: Results and Comparison

- Generate graphs and plots showing fairness under different conditions.
- Compare protocols based on chosen performance metrics.
- Interpret the results and discuss findings.

Weeks 11–12: Finalization

- Organize results, write conclusions, and finalize the project report/book.
- Prepare presentation materials if required.

References & Appendix

GitHub Repository:

<https://github.com/alaajalalabomokh/Fairness-among-TCP-connections.git>

References :

[What is TCP \(Transmission Control Protocol\)? - GeeksforGeeks](#)

[Wayback Machine](#)

[RFC 6582: The NewReno Modification to TCP's Fast Recovery Algorithm](#)

[DCTCP \(DataCenter TCP\) — The Linux Kernel documentation](#)

[TCP Fairness Among Modern TCP Congestion Control Algorithms Including TCP BBR | IEEE Conference Publication | IEEE Xplore](#)